

Functional Test Report

Project: Property Management System

Field	Details
Group Name	Commented Code
Team Members	- Kerem Can Baş - 2321051060 - Abdulahap Akdağ - 2221251046
Course / Institution	Software Verification, Validation, Security / Fatih Sultan Mehmet Vakıf Üniversitesi
Date	May 19, 2026

System Overview

This report documents the functional test suite for the **Software Verification, Validation, Security** project. The system architecture utilizes a React frontend, an Express backend, and a PostgreSQL database. Security is maintained via JWT-based (Bearer Token) authentication.

Functional Test Report	1
Project: Property Management System	1
System Overview	1
🔧 Functional Test Suite: Property Management (MP)	3
🔍 Functional Test Suite: Property Viewing (VIEW)	6
✏️ Functional Test Suite: Property Editing (EDIT)	8
🗑️ Functional Test Suite: Property Deletion (DEL)	10
❤️ Functional Test Suite: Property Favorites (FAV)	11
📋 Functional Test Suite: User Dashboards (MYLIST & MYSAVED)	13
🏠 My Listings (MYLIST)	13
❤️ Saved Properties (MYSAVED)	14
🔍 Functional Test Suite: Property Search (SEARCH)	15
👤 Functional & Security Test Suite: User Registration (REG)	17
1. Functional Registration Flow	17
2. Edge Cases & Data Integrity	18
3. Security & Abuse Prevention	19
🔑 Functional & Security Test Suite: User Login (LOG)	20
1. Functional Login & Validation	20
2. Route Protection & Session Lifecycle	21
3. Security & API Middleware	21
👛 Functional & Security Test Suite: Offer Management (OFF)	23
1. Offer Retrieval & Visibility	23
2. Submission & Validation (The Buyer)	24
3. Responses & Permissions (The Seller)	25

Functional Test Suite: Property Management (MP)

Module: Property Publishing & AI Description

Environment: Frontend (React) / Backend (Express & PostgreSQL)

Authentication: JWT-based (Bearer Token)

Test Case ID	Objective	Test Procedure	Expected Result
TC-FUN C-MP-001	Successful property creation with valid data/images.	1. Navigate to /make-listing . 2. Fill all fields (valid data). 3. Upload 2 images (Multer). 4. Click Submit .	201 Created. Records added to properties and property_images tables. User redirected to dashboard.
TC-FUN C-MP-002	Form validation for missing mandatory fields.	1. Leave Title and Price empty. 2. Fill other fields. 3. Click Submit .	Frontend highlights missing fields. If bypassed via API, backend returns 400 Bad Request . No DB entry.

TC-FUN C-MP-0 03	Unauthorized access prevention (Guest User).	1. Attempt direct URL access to /make-listing . 2. Send POST to /api/property without JWT.	1. Frontend redirects to Login. 2. Backend returns 401 Unauthorized .
TC-FUN C-MP-0 04	AI description generation via Gemini.	1. Upload images. 2. Click Generate AI Description .	Backend calls gemini-2.5-flash . Returns 100-150 word text. UI populates description field.
TC-FUN C-MP-0 05	Image upload limit (Boundary Condition).	1. Fill all text fields. 2. Attempt to upload 11 images. 3. Click Submit .	UI prevents selection OR Backend returns 400/500 (LIMIT_UNEXPECTED_FILE) . DB unchanged.
TC-FUN C-MP-0 06	Data type integrity (Negative Path).	1. Use Playwright API to send POST. 2. Set Price to "abc" or -50000 .	400 Bad Request . Backend validation catches invalid type/sign before SQL execution.

TC-FUN C-MP-0 07	AI Error handling (No context).	1. Leave images empty. 2. Click Generate AI Description.	Backend returns 400 { error: "No images provided..." }. UI displays error toast without crashing.
---------------------------------	------------------------------------	--	--

Functional Test Suite: Property Viewing (VIEW)

Module: Public Property Listing & Detail View

Environment: Frontend (React) / Backend (Express & PostgreSQL)

Access Level: Public (No Authentication Required)

Test Case ID	Objective	Test Procedure	Expected Result
TC-FUN C-VIEW-001	Public access to all-property listings.	1. Log out (clear JWT). 2. Navigate to /properties or send GET to /api/property .	200 OK. Server returns array of properties. UI renders property cards without auth prompts.
TC-FUN C-VIEW-002	View specific property details by ID.	1. Navigate to /property/:id (using a known valid ID).	200 OK. SQL joins/subqueries bundle images correctly. UI renders title, price, and gallery.
TC-FUN C-VIEW-003	Error handling for invalid ID formats.	1. Navigate to a non-numeric route (e.g., /property/abc).	400 Bad Request. Controller passes <code>isNaN()</code> check and rejects request. No server crash; no legacy slice errors.
TC-FUN C-VIEW-004	Response for non-existent Property ID.	1. Navigate to a valid format ID that doesn't exist (e.g., /property/999999).	404 Not Found. Service returns <code>null</code> . UI displays a "Property not found" error state.

TC-FUN C-VIEW- 005	Empty state behavior (Zero properties).	1. Truncate properties table. 2. Navigate to /properties .	200 OK. Server returns empty array [] . UI displays "No properties found" placeholder.
TC-FUN C-VIEW- 006	Property with zero images (Data integrity).	1. View a property detail that has no entries in property_images .	200 OK. PostgreSQL COALESCE or aggregation returns [] . UI shows "No Image Available" without mapping errors.



Functional Test Suite: Property Editing (EDIT)

Module: Listing Modification & Ownership Validation

Environment: Frontend (React) / Backend (Express, PostgreSQL, Zod)

Access Level: Restricted (Owner-Only)

Test Case ID	Objective	Test Procedure	Expected Result
TC-FU NC-EDI T-001	Successful update by the rightful owner.	1. Click "Edit" on owned property. 2. Change Title to "Updated Luxury Villa" and Price to "750000". 3. Click Save .	200 OK. UI reflects changes immediately. DB updated with new values.
TC-FU NC-EDI T-002	Prevent cross-user editing (Idempotency/Security).	1. Log in as User B . 2. Send API update request to a property owned by User A .	403 Forbidden. <code>user_id</code> mismatch detected in controller. DB remains unchanged.
TC-FU NC-EDI T-003	Update attempt on non-existent ID.	1. Authenticate. 2. Send update request to <code>/api/property/99999</code> .	404 Not Found. Service returns <code>null</code> for property check.

TC-FU NC-EDI T-004	Handle non-numeric ID formats.	1. Send update request to <code>/api/property/abc</code> .	400 Bad Request. <code>isNaN()</code> check fails before reaching DB logic.
TC-FU NC-EDI T-005	Unauthorized access (Guest User).	1. Clear JWT. 2. Send update request to valid property ID.	401 Unauthorized. Auth middleware blocks request before controller.
TC-FU NC-EDI T-006	Text length validation (Zod).	1. Submit Title < 5 or > 100 chars. 2. Submit Description < 20 chars.	400 Bad Request. Zod schema throws validation errors. Request never reaches DB.
TC-FU NC-EDI T-007	Numeric integrity validation (Zod).	1. Set Price to <code>0</code> , <code>-100</code> , or <code>"string"</code> .	400 Bad Request. <code>z.coerce.number().positive()</code> catches invalid input.



Functional Test Suite: Property Deletion (DEL)

Module: Listing Removal & Cascade Integrity

Environment: Frontend (React) / Backend (Express & PostgreSQL)

Access Level: Restricted (Owner-Only)

Test Case ID	Objective	Test Procedure	Expected Result
TC-FUN C-DEL-001	Successful deletion by the rightful owner.	1. Navigate to "My Listings". 2. Click Delete and confirm.	200 OK. Property and all related rows (property_images , favorites) are cascaded. UI updates immediately.
TC-FUN C-DEL-002	Prevent unauthorized deletion (Security).	1. Log in as User B . 2. Send DELETE API request for a property owned by User A .	403 Forbidden. Backend detects user_id mismatch. Property remains in DB.
TC-FUN C-DEL-003	Deletion attempt on non-existent ID.	1. Authenticate. 2. Send DELETE request to /api/property/99999 .	404 Not Found. Controller/Service confirms property does not exist.
TC-FUN C-DEL-004	Handle non-numeric ID formats.	1. Send DELETE request to /api/property/abc .	400 Bad Request. isNaN() check fails; no database query is executed.

TC-FUNC C-DEL-0 05	Unauthorized access (Guest User).	1. Clear JWT. 2. Send DELETE request to a valid property ID.	401 Unauthorized. Auth middleware blocks request before it reaches the controller.
-----------------------------------	-----------------------------------	---	---

Functional Test Suite: Property Favorites (FAV)

Module: User Favorites & Personalization

Environment: Frontend (React) / Backend (Express & PostgreSQL)

Access Level: Authenticated Users Only

Test Case ID	Objective	Test Procedure	Expected Result
TC-FUNC -FAV-001	Successfully favorite a property.	1. Log in. 2. Click the Heart button on a listing.	200 OK. Record inserted into favorites junction table. UI heart fills/activates.
TC-FUNC -FAV-002	Successfully remove a favorite.	1. Navigate to "Saved Properties". 2. Click the Unsave button.	200 OK. Record removed from favorites . UI card is removed or heart un-fills.
TC-FUNC -FAV-003	Prevent guest users from favoriting.	1. Clear JWT. 2. Attempt POST to /api/property/:id/favorite .	401 Unauthorized. Auth middleware blocks request before it hits the controller.
TC-FUNC -FAV-004	Handle non-numeric ID formats.	1. Send POST/DELETE to /api/property/abc/favorite .	400 Bad Request. isNaN() check prevents invalid DB queries.

TC-FUNC -FAV-005	Handle duplicate favorite attempts.	1. Favorite Property X. 2. Fire a second POST API request for Property X.	409 Conflict (or idempotent 200 OK). Unique constraint prevents duplicate DB rows.
TC-FUNC -FAV-006	Handle favorite on non-existent ID.	1. Send POST to /api/property/99999/favorite .	404 Not Found . Backend detects missing property; prevents Foreign Key violation crash.

Functional Test Suite: User Dashboards (MYLIST & MYSAVED)

Module: Personalized Property Management & Favorites Retrieval

Environment: Frontend (React) / Backend (Express & PostgreSQL)

Access Level: Restricted (Authenticated User Only)

My Listings (MYLIST)

Test Case ID	Objective	Test Procedure	Expected Result
TC-FUNC-MYLIST-001	Retrieve user-owned listings.	1. Log in as User A . 2. Navigate to "My Listings" or GET /api/property/my-listings .	200 OK. Returns only properties belonging to User A. User B's listings are excluded.
TC-FUNC-MYLIST-002	Prevent guest access to listings.	1. Clear JWT. 2. Attempt GET /api/property/my-listings .	401 Unauthorized. Auth middleware blocks the request before it hits the controller.

♥ Saved Properties (MYSAVED)

Test Case ID	Objective	Test Procedure	Expected Result
TC-FUNC-MY SAVED-001	Retrieve favorited properties.	1. Log in as User A . 2. Navigate to "Saved Properties" or GET /api/property/saved .	200 OK . Returns only properties from the favorites join for User A.
TC-FUNC-MY SAVED-002	Prevent guest access to favorites.	1. Clear JWT. 2. Attempt GET /api/property/saved .	401 Unauthorized . Auth middleware blocks the request.
TC-FUNC-MY SAVED-003	Handle empty saved state.	1. Log in as a user with zero favorites. 2. Navigate to "Saved Properties".	200 OK . Returns <code>[]</code> . UI displays "You haven't saved any properties yet" placeholder.

Functional Test Suite: Property Search (SEARCH)

Module: Global Search & Discovery

Environment: Frontend (React) / Backend (Express & PostgreSQL)

Access Level: Public (No Authentication Required)

Test Case ID	Objective	Test Procedure	Expected Result
TC-FUNC-SEARCH-001	Partial string matching (Title/Address).	1. Navigate to search. 2. Query for "cozy". 3. GET <code>/api/property/search?q=cozy</code> .	200 OK. Returns properties where "cozy" appears in the title or address.
TC-FUNC-SEARCH-002	Case-insensitive matching.	1. Query for "luxury penthouse" (lowercase). 2. Database contains "LUXURY PENTHOUSE".	200 OK. PostgreSQL ILIKE ignores casing and returns the match successfully.
TC-FUNC-SEARCH-003	Empty result handling (Zero matches).	1. Search for a term with no matches (e.g., "Castle").	200 OK. Server returns <code>[]</code> . UI displays "No properties found for 'Castle'" message.
TC-FUNC-SEARCH-004	Handle missing query parameter.	1. Fire GET request to <code>/api/property/search</code> without the <code>?q=</code> param.	400 Bad Request. Controller detects missing <code>q</code> and returns "Search term is required".

Functional & Security Test Suite: User Registration (REG)

Module: User Onboarding & Account Security

Environment: Frontend (React) / Backend (Express, PostgreSQL, express-rate-limit)

Access Level: Public (Guest)

1. Functional Registration Flow

Test Case ID	Objective	Test Procedure	Expected Result
TC-REG-001	Valid user registration.	1. Enter valid Name, Email, and Password. 2. Click Register .	201 Created. New record in users table. Redirect to Login/Home.
TC-REG-002	Prevent duplicate emails.	1. Attempt to register with an email already in the DB.	409 Conflict. UI displays "Email already in use." No crash.
TC-REG-004/013	Mandatory field validation.	1. Leave Name, Email, or Password blank. 2. Attempt submission.	Submit disabled or inline "Field is required" errors.
TC-REG-005	Invalid email format check.	1. Test plainaddress or user@.com .	UI prevents submission; Backend returns 400 Bad Request .

TC-REG-006	Password confirmation check.	1. Enter mismatched passwords in "Confirm Password".	Form blocked; UI highlights mismatch.
TC-REG-007	Password complexity enforcement.	1. Attempt weak password (e.g., 12345).	400 Bad Request. UI shows specific rules (e.g., 8+ chars, 1 symbol).

2. Edge Cases & Data Integrity

Test Case ID	Objective	Test Procedure	Expected Result
TC-RE G-008	Whitespace sanitization.	1. Enter user@ex.com (with spaces).	201 Created. Backend trims spaces; DB saves user@ex.com.
TC-RE G-009	Case-insensitive email check.	1. Try USER@EX.COM when user@ex.com exists.	409 Conflict. Backend normalizes emails to lowercase.
TC-RE G-010	Max field length limits.	1. Enter Name > 255 characters.	HTML maxlength blocks OR Backend returns 400 Bad Request .

3. Security & Abuse Prevention

Test Case ID	Objective	Test Procedure	Expected Result
TC-RE G-003	SQL Injection prevention.	1. Enter <code>Robert'); DROP TABLE users;--</code> in Name.	201 Created. Name saved literally. DB structure remains intact.
TC-RE G-011	XSS payload handling.	1. Enter <code><script>alert('XSS')</script></code> in Name.	201 Created. React escapes string in UI; script does not execute.
TC-RE G-012	Rate Limiting (Spam/DoS).	1. Send 10+ rapid POSTs via Playwright API.	429 Too Many Requests. Server blocks IP after defined limit.

Functional & Security Test Suite: User Login (LOG)

Module: Authentication, Session Management, and Middleware

Environment: Frontend (React) / Backend (Express & PostgreSQL)

Access Level: Public (Guest) to Private (Authenticated)

1. Functional Login & Validation

Test Case ID	Objective	Test Procedure	Expected Result
TC-LO G-001	Successful login & JWT storage.	1. Enter valid credentials. 2. Submit.	200 OK. JWT saved to localStorage . UI state updates. Redirect to dashboard.
TC-LO G-002	Handle incorrect password.	1. Enter correct email + wrong password.	401 Unauthorized. No token issued. UI shows "Invalid credentials."
TC-LO G-005	Invalid email format (Zod).	1. Enter not-an-email and a password.	Submission blocked. UI displays "Invalid email format." No API call.
TC-LO G-006	Empty field validation.	1. Attempt login with empty fields.	Submission blocked. "Field is required" warnings appear.
TC-LO G-007	Whitespace & Casing (Sanitization).	1. Enter TEST@example.com + password.	200 OK. Zod trims/lowercases before API call. Success.

2. Route Protection & Session Lifecycle

Test Case ID	Objective	Test Procedure	Expected Result
TC-LOG-003	Frontend Route Guard.	1. Try to visit <code>/my-listings</code> while logged out.	Redirected to <code>/login</code>. Protected route is inaccessible.
TC-LOG-010	Session persistence.	1. Log in. 2. Perform a hard page reload.	Still logged in. React state re-hydrates from <code>localStorage</code> .
TC-LOG-011	Logout functionality.	1. Click "Logout".	State Cleared. Token removed from <code>localStorage</code> . Redirect to Login.
TC-LOG-012	Handle expired JWT.	1. Trigger API call with an expired token.	401 Unauthorized. Frontend clears local state and forces redirect.

3. Security & API Middleware

Test Case ID	Objective	Test Procedure	Expected Result
TC-LOG-004	Tampered JWT rejection.	1. Send request with <code>Bearer fake.token.here</code> .	401/403. Signature check fails. No private data returned.

TC-LO G-008	Non-existent user check.	1. Login with an unregistered email.	401 Unauthorized. Generic "Invalid credentials" error to prevent email sniffing.
TC-LO G-009	SQL Injection in Login.	1. Enter ' OR 1=1 -- in fields.	401 Unauthorized. Sanitized inputs prevent bypass. No DB crash.
TC-LO G-013	Missing Auth Header.	1. Call protected API without a header.	401 Unauthorized. Middleware blocks execution.
TC-LO G-014	Malformed Auth Header.	1. Send Authorization: <token> (No "Bearer").	401 Unauthorized. Middleware identifies bad header format.

Functional & Security Test Suite: Offer Management (OFF)

Module: Offers, Seller Dashboards, and Negotiation Logic

Environment: Frontend (React) / Backend (Express & PostgreSQL)

Access Level: Authenticated Buyers and Sellers

1. Offer Retrieval & Visibility

Test Case ID	Objective	Test Procedure	Expected Result
TC-FUNC-OFF-001	Get offers for a specific property.	1. Authenticate. 2. GET <code>/api/offer/property/:id</code> .	200 OK. Returns array of offers + <code>buyer_name</code> .
TC-FUNC-OFF-002	Seller's "Received Offers" dashboard.	1. Log in as Seller. 2. GET <code>/api/offer/received</code> .	200 OK. Aggregates all offers across all properties owned by Seller.
TC-FUNC-OFF-003	Guest access prevention.	1. Clear JWT. 2. Attempt GET on offer endpoints.	401 Unauthorized. Middleware blocks access.
TC-FUNC-OFF-012	Handle zero-offer state.	1. GET offers for a new property.	200 OK. Returns <code>[]</code> . UI shows "No offers yet."

2. Submission & Validation (The Buyer)

Test Case ID	Objective	Test Procedure	Expected Result
TC-FUNC-OFF-004	Valid offer submission.	1. Enter amount (450000). 2. Submit.	201 Created. Record added to offers . Success toast shown.
TC-FUNC-OFF-008	Invalid amount validation.	1. Submit amount -5000 or 0 .	400 Bad Request. Validation catches error; DB unchanged.
TC-FUNC-OFF-009	Non-existent Property ID.	1. Submit offer for a fake UUID.	404/400. Foreign key or logic catch. No orphan records.
TC-FUNC-OFF-010	Self-Offer Prevention.	1. Seller attempts to offer on own listing.	403 Forbidden. Business logic blocks "self-buying."
TC-FUNC-OFF-011	Spam/Duplicate prevention.	1. Submit second offer while one is PENDING.	409 Conflict. One active offer per buyer/property pair.

3. Responses & Permissions (The Seller)

Test Case ID	Objective	Test Procedure	Expected Result
TC-FUNC-OFF-006	Valid Status Update (Accept/Reject).	1. Seller clicks "Accept". 2. PATCH /status with "ACCEPTED".	200 OK. Status updated. UI badge changes color.
TC-FUNC-OFF-007	Attacker/Hijack Prevention.	1. Non-owner attempts to PATCH an offer status.	403 Forbidden. <code>getOfferOwnership</code> check fails.
TC-FUNC-OFF-013	Immutable state enforcement.	1. Attempt to REJECT an already ACCEPTED offer.	400 Bad Request. Finalized states cannot be changed.